



Universidade do Minho
Licenciatura em Ciências da Computação

Unidade Curricular de Computação Gráfica

Ano Letivo de 2023/2024

Curvas, Superfícies Cúbicas e VBOs – Fase 3

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Abril, 2024

CG

Data de Recepção	
Responsável	
Avaliação	
Observações	

Curvas, Superfícies Cúbicas e VBOs – Fase 3

Grupo 12

Bruna Micaela Rodrigues Araújo a84914,

Hugo Alexandre Peres Ferreira a100082,

Nuno Filipe Norberto Gonçalves de Oliveira a53971

Abril, 2024

Introdução

O presente relatório é referente à terceira fase de um projeto desenvolvido no âmbito da unidade curricular de Computação Gráfica, tendo como propósito obter uma representação dinâmica e mais realista do sistema solar. Para tal foram implementadas curvas, superfícies cúbicas e VBOs.

No *generator* foi proposta a conversão de um dado ficheiro *.patch* em ficheiro *.3d* a partir de um conjunto de *patches de Bezier*.

No *engine* foram acrescentadas transformações com tempo. No caso da translação podem agora ser definidas animações através de curvas de *Catmull-Rom*; no caso da rotação passa a ser possível efetuar rotações de 360° em torno de um dado eixo; o modo de desenho de primitivas foi alterado para VBOs.

Ao longo deste relatório irão ser explicadas, de forma detalhada, todas as decisões tomadas.

Índice

Introdução	i
Índice	ii
Índice de Figuras	iii
1. <i>Generator</i>	1
1.1. <i>Patches de Bezier</i>	1
2. <i>Engine</i>	3
2.1. Leitura do ficheiro <i>XML</i>	3
2.2. Implementação de <i>VBOs</i>	4
2.3. Transformações com tempo	5
3. Sistema Solar	7
4. Resultados	8
4.1. Capturas referentes aos testes fornecidos	8
4.2. Capturas do Sistema Solar dinâmico	9
5. Conclusão	10

Índice de Figuras

Figura 1 – test_3_2	8
Figura 2 – test_3_1	8
Figura 3 – test_3_1 (com diferença de 2 segundos da Figura 2)	8
Figura 4 – solar_system (instante 11)	9
Figura 5 – solar_system (instante 53)	9
Figura 6 – solar_system (instante 134)	9

1. Generator

1.1. Patches de Bezier

Em ordem a ser possível criar modelos baseados em *patches de Bezier*, a aplicação *generator* foi então atualizada com esse propósito, através da implementação das seguintes funções:

- *loadPatch* : função que permite ler um ficheiro *.patch*. Esta função lê do ficheiro as informações necessárias para a construção dos *patches*, tais como o número de *patches*, os índices para a construção de cada *patch*, o número de pontos de controlo e os pontos de controlo a utilizar. Após a obtenção destas informações são então guardados os pontos de controlo (conjunto de *float's* x, y, z) que dizem respeito a cada *patch*.

Abaixo segue o ciclo da construção dos *patches*.

```
for (int i = 0; i < numberOfPatches; i++)
{
    vector<vector<float>> patch;
    vector<int> indices = controlPointsIndices[i];
    for (int indice : indices)
    {
        patch.push_back(controlPoints[indice]);
    }
    patchInfo.push_back(patch);
}
```

- *createPrimitive* : função onde são calculados os pontos da superfície de *Bezier* de cada *patch* (através da função *bezierSurfacePoint*) e guardados os pontos referentes aos triângulos necessários para desenhar a superfície no ecrã num ficheiro *.3d*.

- *bezierSurfacePoint* : função que calcula o ponto de uma superfície de Bezier de acordo com a fórmula abaixo apresentada. É necessário fazer este cálculo para cada uma das três componentes X, Y e Z que representam um ponto. Quanto maior o nível de *tessellation* maior será o detalhe com que a primitiva será desenhada.

$$M = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = M^T$$

$$P(u, v) = [u^3 \ u^2 \ u \ 1] M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

2. Engine

2.1. Leitura do ficheiro XML

Uma das funcionalidades propostas nesta fase consiste na possibilidade de fazer transformações “animadas” baseadas em translações ao longo de curvas cúbricas de *Catmull-Rom* e, rotações de 360° à volta de um determinado eixo. Para que tal fosse possível, foram atualizadas a função de leitura do ficheiro XML (função *addGroup*) e a estrutura de dados que diz respeito às transformações geométricas, a fim de possibilitar a gravação de novos parâmetros fornecidos no ficheiro XML.

```
struct Transform
{
    int type;
    float angle;
    Point coords;
    float time = 0;
    int align = 0;
    vector<Point> points;
};
```

Foram, portanto, acrescentados novos campos à estrutura *Transform*:

- *time* : contém o número de segundos necessários para percorrer uma curva de *Catmull-Rom* ou o tempo necessário para fazer uma rotação de 360° ;
- *align* : permite saber se é necessário alinhar a primitiva geométrica com a curva – 0 caso não seja necessário, 1 caso contrário;
- *points* : contém os pontos que definem a curva de Catmull-Rom.

2.2. Implementação de VBOs

Nesta fase foi implementado o desenho dos modelos através de VBOs com o propósito de aumentar a eficiência da renderização de cenas e do programa.

Foi tomada a decisão de utilizar vários *buffers* para guardar a informação dos respetivos modelos. Para concretizar esta ideia, inicialmente conta-se o número de primitivas a desenhar e aloca-se o espaço de memória necessário

```
// Get number of models to draw
modelsCount(conf.group, &nModels);
// Allocate memory for buffers
buffers = (GLuint *)malloc(sizeof(GLuint) * nModels);
```

Posteriormente, através da função *modelsToBuffers*, e para cada primitiva, copia-se dos *models* para os *buffers* o conteúdo referente aos pontos necessários para desenhar a respetiva primitiva. Por forma a fazer um mapeamento entre primitiva e *buffer*, utiliza-se a variável *buffersIndex*.

Sendo necessário saber qual o tamanho de cada *buffer* para então ser possível copiar os dados para a placa gráfica, foi criado também um vetor para armazenar essa informação.

```
// Variáveis globais - buffers e vector com tamanho dos buffers
GLuint *buffers;
vector<GLuint> buffersSize;
```

2.3. Transformações com tempo

○ Rotação:

Nesta fase do presente projeto, no ficheiro *XML*, o ângulo (parâmetro da rotação) pode ser substituído pelo tempo, ou seja, pelo número de segundos necessários para completar uma volta em torno do eixo especificado.

A fim de diferenciar as rotações “estáticas” das rotações “animadas”, o primeiro passo a dar é verificar se o valor do tempo fornecido é maior do que 0. Em seguida, calcula-se o valor do ângulo em cada intervalo de tempo. Há a necessidade de multiplicar o valor por 360 por se tratar de uma rotação completa (360°) e dividir por 1000 para a unidade passar de milissegundos para segundos.

```
case 1:
    if (it->time > 0)
    {
        it->angle = (currentTime - startTime) * 360.0 / 1000.0 / it->time;
    }
    glRotatef(it->angle, it->coords.px, it->coords.py, it->coords.pz);
    break;
```

○ Translação:

Para as translações, e com o propósito de definir uma curva cúbica de *Catmull-Rom*, vai ser passado um conjunto de pontos e o número de segundos necessários para a percorrer. O objetivo principal é ser possível fazer animações baseadas nestas curvas.

Além disso, nesta transformação foi incluído também um parâmetro *align* para especificar se o objeto deverá ou não estar alinhado com a curva.

O cálculo dos pontos da curva em que se encontra o objeto num dado instante de tempo é feito recorrendo às funções *getGlobalCatmullRomPoint* e *getCatmullRomPoint*.

Para calcular um ponto numa curva de *Catmull-Rom*, dados quatro pontos de controlo, é usada a seguinte fórmula:

$$p(t) = [t^3 \ t \ t \ 1] \begin{bmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & -0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} .$$

Caso seja preciso alinhar o objeto com a curva, há a necessidade de ser definida a orientação dos eixos do seu sistema de coordenadas cada vez que for calculada uma nova posição. Abaixo seguem os cálculos que devem ser efetuados.

$$\vec{X}_t = p'(t)$$

$$\vec{Z}_t = \vec{X}_t \times \vec{Y}_{t-1}$$

$$\vec{Y}_t = \vec{Z}_t \times \vec{X}_t$$

$p'(t)$: vector tangente à curva

$$M = \begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

M : matriz de rotação

Considerou-se que a orientação inicial do eixo Y do sistema de coordenadas do objeto é

$$Y_0 = (0, 1, 0) .$$

3. Sistema Solar

Tendo agora como por objetivo “animar” as rotações, e para desenhar o Sistema Solar, no ficheiro *XML*, o parâmetro *angle* (correspondente ao ângulo) foi alterado para *time* (tempo).

Um dos requisitos desta fase do projeto era a implementação de um cometa com uma trajetória definida, utilizando curvas de *Catmull-Rom*. Para concretizar este objetivo foi definido o grupo abaixo apresentado, onde são utilizados 12 pontos de controlo e é especificado que demora 15 segundos a percorrer toda a curva.

```
<group>
  <transform>
    <rotate angle="10" x="0" y="0" z="1" />
    <translate time = "15" align="true"> <!-- O campo align
diz se o objecto deve ser orientado na curva -->
      <point x = "0" y = "0" z = "50" />
      <point x = "25" y = "0" z = "43.3" />
      <point x = "43.3" y = "0" z = "25" />
      <point x = "60" y = "0" z = "0" />
      <point x = "43.3" y = "0" z = "-25" />
      <point x = "25" y = "0" z = "-43.3" />
      <point x = "0" y = "0" z = "-50" />
      <point x = "-25" y = "0" z = "-43.3" />
      <point x = "-43.3" y = "0" z = "-25" />
      <point x = "-60" y = "0" z = "0" />
      <point x = "-43.3" y = "0" z = "25" />
      <point x = "-25" y = "0" z = "43.3" />
    </translate>
    <rotate angle="-90" x="1" y="0" z="0" />
    <scale x="0.5" y="0.5" z="0.5" />
  </transform>
  <models>
    <model file="bezier_10.3d" /> <!-- generator patch
teapot.patch 10 bezier_10.3d -->
  </models>
</group>
```

4. Resultados

4.1. Capturas referentes aos testes fornecidos

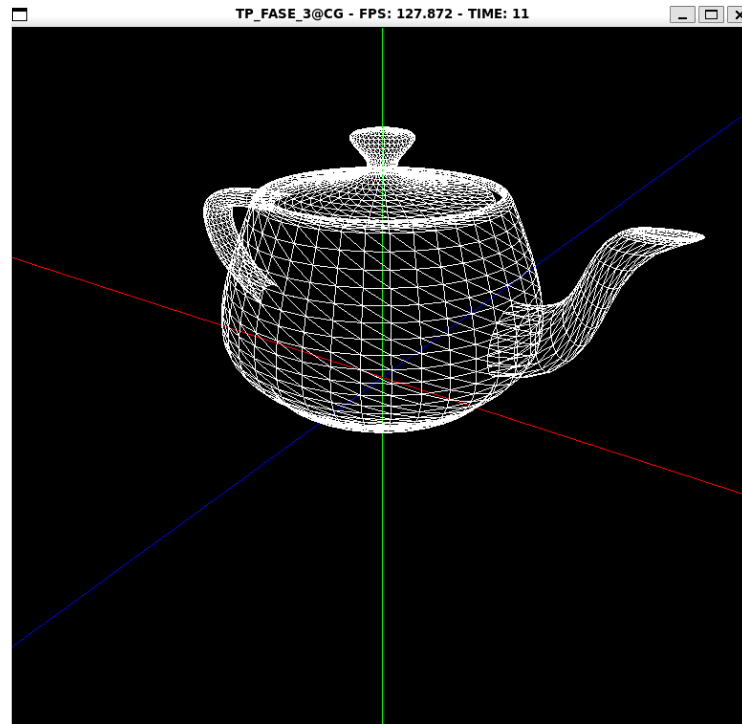


Figura 1 - test_3_2

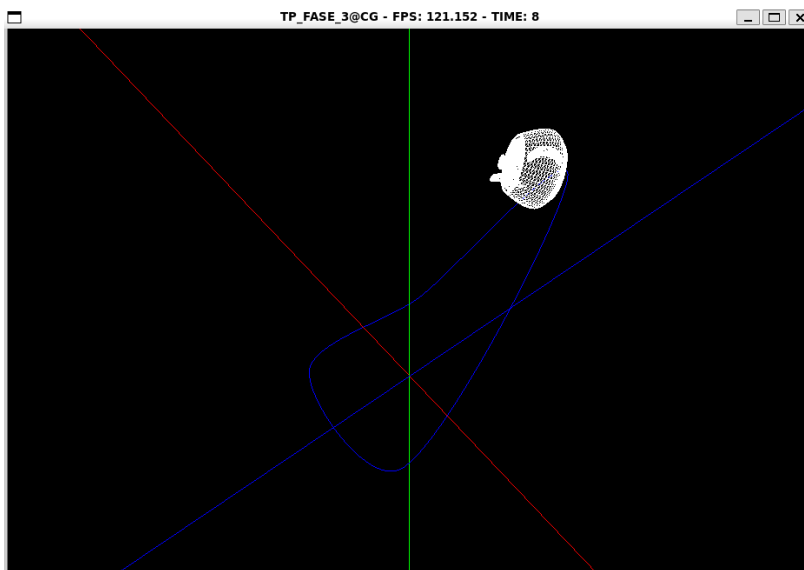


Figura 2 - test_3_1

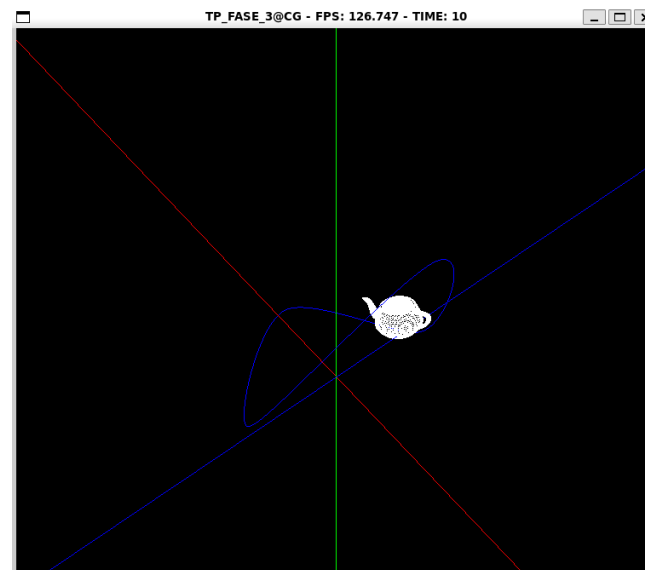


Figura 3 - test_3_1 (com diferença de 2 segundos da Figura 2)

4.2. Capturas do Sistema Solar dinâmico

Nesta secção são apresentadas várias figuras capturadas em instantes diferentes, a fim de demonstrar o movimento dinâmico do Sistema Solar implementado.

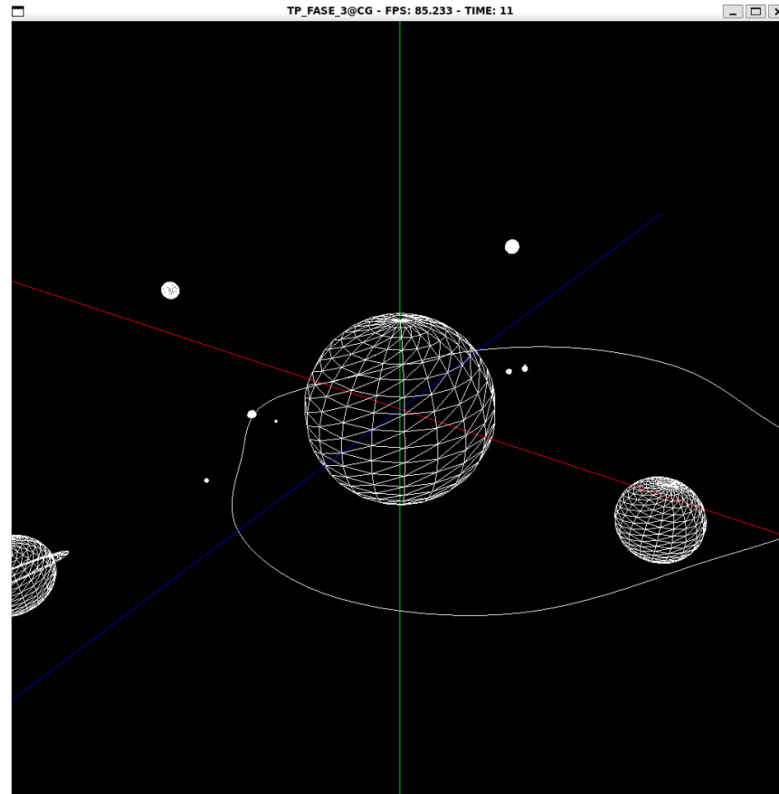


Figura 4 - solar_system (instante 11)

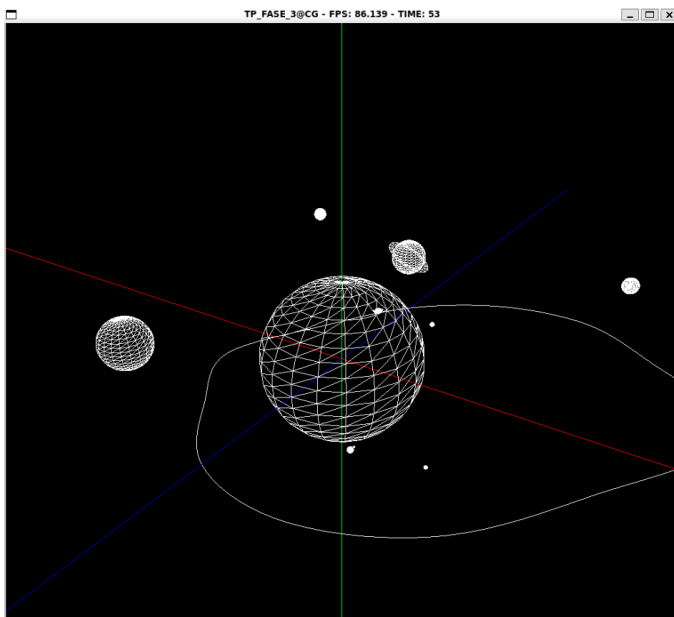


Figura 5 - solar_system (instante 53)

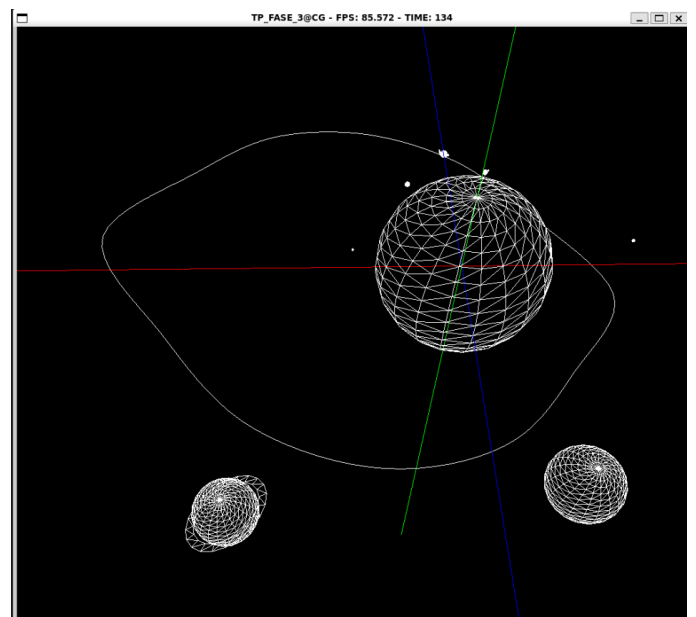


Figura 6 - solar_system (instante 134)

5. Conclusão

Esta terceira fase do projeto de Computação Gráfica mostrou avanços significativos, apontando a uma representação mais dinâmica do sistema solar.

No *generator* foi implementada a conversão de ficheiros *.patch* em ficheiros *.3d*, permitindo assim a criação de modelos em *patches* de *Bezier*.

Por sua vez, no *engine*, foram implementados com sucesso *VBOs* e introduzidas transformações animadas, incluindo translações ao longo de curvas cúbicas de *Catmull-Rom* e rotações de 360° em torno de eixos específicos.

As transformações com tempo foram bem implementadas, garantindo assim animações suaves e realistas dos objetos.

Em suma, considera-se que os requisitos propostos foram cumpridos e as dificuldades sentidas foram superadas com resiliência.